ragent Spec Management Manual

This guide explains how to use ragent's built-in spec management system to perform specification-driven design and planning. The `/spec` command family lets you create EARS-notation specifications, generate implementation plans, validate compliance, manage lifecycle status, and drive implementation — all from the interactive TUI.

The commands are exposed via the `/spec` slash command family (TUI) and the `spec_*` LLM tools (`spec_read`, `spec_list`, `spec_search`, `spec_coverage`, `spec_task_update`); the spec lifecycle is managed by the `ragent-specs` crate.

> **Scope:** `/spec` slash commands, EARS requirements notation, the SDD (Spec-Driven Development) workflow, lifecycle status transitions, task management, implementation orchestration, JTBD analysis, production feedback, and configuration. For research system commands, see `docs/howtos/research.md`.

---

1. Purpose and capabilities

The spec management system brings structure to feature design and implementation planning. Instead of ad-hoc prompts, you create formal specifications using EARS (Easy Approach to Requirements Syntax) notation, generate implementation plans with dependency-ordered tasks, validate compliance, and orchestrate implementation.

What it does

- **EARS specification** — creates `SPEC.md` files using EARS notation with five template types (ubiquitous, event-driven, state-driven, optional, unwanted).
- **Implementation planning** — generates `PLAN.md` with a task table containing IDs, titles, linked requirements, effort estimates, priorities, and dependency chains.
- **Manual test planning** — generates `TESTPLAN.md` with human-readable test cases including preconditions, steps, test data, and expected results.
- **EARS validation** — validates spec compliance including section detection, requirement numbering, EARS template matching, ambiguity detection, contradiction detection, and gap analysis.
- **Lifecycle management** — tracks spec status through seven stages with enforced transition rules.
- **Task management** — tracks individual task status (pending, in_progress, completed, blocked) within a plan.
- **Implementation orchestration** — executes plan tasks in dependency order, injecting per-task prompts into the agent session.
- **JTBD analysis** — performs Jobs-To-Be-Done analysis on existing specs, writing structured job statements with requirement traceability.
- **Production feedback** — appends advisory notes to `FEEDBACK.md` that are surfaced during plan regeneration.
- **SDD workflow separation** — separates specification (`/spec specify`) from planning (`/spec plan`), matching the Spec-Driven Development methodology.
- **Research integration** — links research artifacts to specs via `--from-research`, embedding a `## Related Research` section and YAML frontmatter linkage.
- **Branch-per-spec** — optionally creates a dedicated git branch (`spec/<specname>`) for each specification.

What it does not do

- It does not auto-generate code — `/spec impl` drives the agent through tasks, but the agent performs the actual implementation.
- It does not manage CI/CD pipelines — use the VCS tools for that.
- It does not replace issue tracking — use the GitHub/GitLab tools for external issue management.

---

2. Quick start

Open ragent and create your first spec:

```
/spec create websocket Add a real-time collaborative editing feature using WebSockets
```

The TUI will:

1. Create a new spec directory at `specs/websocket/`.
2. Generate `SPEC.md` (EARS requirements), `PLAN.md` (implementation plan), and `TESTPLAN.md` (manual test plan) using the explore agent.
3. Stream progress to the message window.
4. When finished, validate the spec:

```
/spec validate websocket
```

List all specs:

```
/spec list
```

Check requirement coverage:

```
/spec coverage websocket
```

Start implementation:

```
/spec impl websocket
```

---

3. What specs are

A spec is a self-contained directory under `specs/<id>/`. **Only `SPEC.md` is required**: a directory holding just a `SPEC.md` is discovered by `/spec list`, `/spec search` and the `spec_*` tools, and is a valid target for `/spec update <name>`, `/spec plan <name>` and `/spec validate <name>`. This is what makes a hand-authored or partially-generated spec usable before its plan exists. Everything below `SPEC.md` in the tree is optional and derived.

```
specs/
  websocket/
      SPEC.md          # EARS requirements specification
      PLAN.md          # implementation plan with task table
      TESTPLAN.md      # manual test plan with test cases
      TASKS.md         # ordered task list (from /spec tasks)
      quickstart.md    # quickstart validation scenarios (from /spec tasks)
      JTBD.md          # Jobs-To-Be-Done analysis (from /spec jtbd)
      FEEDBACK.md      # production feedback notes (from /spec feedback)
```

```
    data-model.md    # data entity model (from /spec plan, if enabled)
    contracts/       # API contract definitions (from /spec plan, if enabled)
auth-refactor/
    SPEC.md
    PLAN.md
    TESTPLAN.md
```

File descriptions

File	Created by	Description
`SPEC.md`	`/spec create`, `/spec specify`	EARS requirements specification with YAML frontmatter
`PLAN.md`	`/spec create`, `/spec plan`, `/spec update`	Implementation plan with task table
`TESTPLAN.md`	`/spec create`, `/spec update`	Manual test plan with test cases
`TASKS.md`	`/spec tasks`	Ordered task list derived from PLAN.md
`quickstart.md`	`/spec tasks`	Quickstart validation scenarios from SPEC.md requirements
`JTBD.md`	`/spec jtbd`	Jobs-To-Be-Done analysis with coverage matrix
`FEEDBACK.md`	`/spec feedback`	Production feedback notes (advisory)
`data-model.md`	`/spec plan` (if `sdd.data_model` enabled)	Data entity model with relationships and constraints
`contracts/`	`/spec plan` (if `sdd.contracts` enabled)	API contract definitions

Naming rules

Spec IDs must be valid directory names: alphanumeric characters, hyphens, and underscores. The `SpecId` type validates this at creation time.

Valid IDs: `websocket`, `auth-refactor`, `q3-review`, `api-v2`, `auth_refactor`

Invalid IDs: `WebSocket` (uppercase), `api.v2` (dots)

---

4. The EARS notation

EARS (Easy Approach to Requirements Syntax) is a structured template for writing unambiguous requirements. ragent's spec system uses five EARS template types:

Template	Pattern	Use case
Ubiquitous	`The <system> shall <response>.`	Always-on requirements
Event-driven	`When <trigger>, the <system> shall <response>.`	Reactive requirements

Template	Pattern	Use case
State-driven	`While <precondition>, the <system> shall <response>.`	Conditional state requirements
Optional	`Where <feature> is included, the <system> shall <response>.`	Feature-gated requirements
Unwanted	`If <trigger>, the <system> shall <response>.`	Error/exception handling

Each spec should use at least one of each template type. Requirements are numbered as `FR-001`, `FR-002`, etc. (functional requirements) or `NFR-001`, `NFR-002`, etc. (non-functional requirements).

Example SPEC.md structure

```
---
status: draft
audit:
  - { time: 1787099746, from: "none", to: "draft", actor: "system" }
---

# Specification: websocket - Real-time Collaborative Editing

## Overview

This specification defines a WebSocket-based real-time collaborative editing
feature for the application.

## Requirements

### FR-001
The system shall establish a WebSocket connection within 2 seconds of user
action. (Ubiquitous)

### FR-002
When a user edits a document, the system shall broadcast the change to all
connected clients within 100ms. (Event-driven)

### FR-003
While the connection is unstable, the system shall queue edits and replay
them when connectivity is restored. (State-driven)

### FR-004
Where CRDT support is included, the system shall merge concurrent edits
without conflict. (Optional)

### FR-005
```

```
If the WebSocket server is unreachable, the system shall display a
reconnection prompt with a retry button. (Unwanted)
```

---

5. The `/spec` command family

All commands are entered in the TUI input box.

5.1 `/spec help`

Show the command reference table with all subcommands, arguments, and descriptions.

```
/spec help
```

5.2 `/spec create <specname> <feature> [--from-research <name>]`

Create a new spec with `SPEC.md`, `PLAN.md`, and `TESTPLAN.md` in one step. This is the all-in-one command for getting a spec off the ground quickly.

The `--from-research <name>` flag links a research artifact by adding a `research:` field to the YAML frontmatter and a `## Related Research` section to the SPEC.md body.

```
/spec create websocket Add a real-time collaborative editing feature using WebSockets
/spec create auth-v2 Implement OAuth2 authentication with PKCE flow
/spec create data-export Add CSV and JSON export for all list views --from-research export-format
```

5.3 `/spec specify <specname> <feature> [--from-research <name>]`

Create a `SPEC.md` only — no `PLAN.md` or `TESTPLAN.md`. This separates the specification stage from the planning stage, matching the SDD workflow. The agent inserts `[NEEDS CLARIFICATION: <question>]` markers wherever a requirement is ambiguous.

After reviewing and resolving clarification markers, use `/spec plan` to generate the implementation plan.

When `sdd.branch_per_spec` is enabled in `ragent.json`, this command also creates a `spec/<specname>` git branch.

```
/spec specify websocket Add a real-time collaborative editing feature
/spec specify auth-v2 Implement OAuth2 with PKCE --from-research oauth-study
```

5.4 `/spec plan <spec-id> <tech-context>`

Generate or regenerate `PLAN.md` from an existing `SPEC.md` using the provided technology context as guidance. This is the second stage of the SDD workflow after `/spec specify`.

The technology context is a free-text description that informs technology choices and rationale documented in the plan. If a `PLAN.md` already exists, task statuses for unchanged task IDs are preserved.

When `sdd.data_model` is enabled, the prompt also instructs the agent to generate an optional `data-model.md` artifact. When `sdd.contracts` is enabled, it instructs the agent to generate an optional `contracts/` directory. When `FEEDBACK.md` exists, its notes are surfaced in the prompt.

```
/spec plan websocket Use tokio-tungstenite for WebSocket server, Redis for pub/sub
/spec plan auth-v2 Use oauth2 crate with PKCE, store tokens in SQLite via rusqlite
/spec plan api-v2 Migrate from REST to gRPC using tonic, protobuf schemas in proto/ directory
```

5.5 `/spec update <spec-id>`

Regenerate `PLAN.md` and `TESTPLAN.md` from an edited `SPEC.md`. This command re-reads the current `SPEC.md` (which you may have manually edited) and regenerates the plan and test plan to match. It does NOT modify `SPEC.md`.

Task statuses for unchanged task IDs are preserved, and frontmatter keys that the lifecycle manager does not model itself (for example a `research:` linkage added by `--from-research`) survive regeneration — `update_frontmatter` rewrites only the status and audit fields and keeps every other key, with values that can contain YAML-special characters properly quoted.

```
/spec update websocket
/spec update auth-v2
```

5.6 `/spec add <spec-id> <feature>`

Incrementally add requirements to an existing spec and update its plan. The agent reads the existing `SPEC.md` and `PLAN.md`, generates incremental requirements and tasks for the new feature, and inserts them without modifying existing sections.

```
/spec add websocket Add presence indicators showing who is currently viewing each document
/spec add auth-v2 Add refresh token rotation with sliding expiration
```

5.7 `/spec validate [spec-id]`

Validate EARS compliance. Without a spec ID, validates all specs.

Validation checks include:

- **Required section detection** — verifies `## Requirements` section exists
- **Requirement numbering** — checks for gaps (e.g. FR-001, FR-003 missing FR-002) and duplicates
- **EARS template matching** — verifies each requirement matches a known EARS pattern
- **Status value validation** — checks the YAML frontmatter status is a valid `SpecStatus`
- **PLAN.md completeness** — verifies the plan has a `## Tasks` table
- **Clarification markers** — detects `[NEEDS CLARIFICATION]` markers (when `sdd.clarification_markers` is enabled)
- **Ambiguity detection** — flags vague terms, undefined acronyms (when `sdd.consistency_checks` is enabled)
- **Contradiction detection** — finds opposing requirement claims (when `sdd.consistency_checks` is enabled)
- **Gap analysis** — identifies missing testable criteria (when `sdd.consistency_checks` is enabled)
- **Phase -1 gates** — validates pre-implementation gate checkboxes (when `sdd.phase_minus_one_gates` is enabled)

```
/spec validate websocket
/spec validate              # validate all specs
```

5.8 `/spec list [--status <status>] [--prefix <prefix>]`

List all specs with optional filtering.

Filters:

- `--status <status>` — filter by lifecycle status (draft, in_review, approved, in_progress, implemented, verified, archived)
- `--prefix <prefix>` — filter by spec ID prefix (case-insensitive)
- `--archived` — include archived specs (excluded by default); `--all` is accepted as a synonym

By default, specs are sorted by most recently modified. Other sort options: by status (lifecycle order), by ID (alphanumeric), or by title.

```
/spec list
/spec list --status approved
/spec list --status=approved
/spec list --prefix auth
/spec list --archived
```

The `--status <s>` and `--status=<s>` forms behave identically. Argument handling is strict: an unknown status or flag is rejected with the valid status list, a flag followed by another option (`--prefix --archived`) or an empty `--status=` is reported as a missing value, and neither case falls back to listing every spec.

5.9 `/spec search <query>`

Full-text search across all specs. Returns matching spec IDs, titles, and context snippets around the matches.

```
/spec search websocket
/spec search authentication OAuth
/spec search "error handling"
```

5.10 `/spec status <spec-id> [<new-status>]`

Show the current status of a spec, or transition it to a new status.

Status transitions are enforced — you cannot skip stages. See section 6 for the full transition graph.

```
/spec status websocket                  # show current status
/spec status websocket in_review        # transition to in_review
/spec status websocket approved         # transition to approved
```

5.11 `/spec task <spec-id> [<task-id>] [<new-status>]`

List tasks, show a single task, or update a task's status.

```
/spec task websocket                      # list all tasks
/spec task websocket T-003                # show task T-003
/spec task websocket T-003 in_progress    # mark T-003 as in_progress
/spec task websocket T-003 completed      # mark T-003 as completed
/spec task websocket T-005 blocked        # mark T-005 as blocked
```

5.12 /spec activate <spec-id>

Activate a spec for context injection. When a spec is active, its SPEC.md content is injected into the agent's system prompt so the agent follows the spec's requirements during implementation.

```
/spec activate websocket
```

5.13 /spec deactivate

Deactivate the currently active spec. No more spec context will be injected into agent prompts.

```
/spec deactivate
```

5.14 /spec coverage <spec-id>

Show a requirement coverage report. This displays which requirements have linked tasks and which do not, helping identify gaps in the implementation plan.

The report (rendered by a shared `Spec::coverage_report()` implementation used by both the TUI command and the `spec_coverage` agent tool, so both consumers always agree on format) contains:

- **Overall Coverage** — the percentage of linked requirements whose tasks are all completed.
- **Requirements section** — one line per requirement with a `[ok]` / `[ ]` status symbol, the requirement ID and text, and either "(N of M linked tasks completed)" or "(no linked tasks)".
- **Tasks section** — one line per task with its status symbol (`[ok]`, `[..]`, `[ ]`, `[!!]` for completed, in-progress, pending, blocked), the task ID and title, its status word, and its linked requirement IDs (or `(unlinked)`).

```
/spec coverage websocket
```

5.15 /spec impl <spec-id> [--task <ID>] [--dry-run]

Implement a spec by executing its PLAN.md tasks in dependency order. The runner reads the plan, resolves dependency order, and constructs per-task prompts that drive the agent through implementing each task sequentially.

Flags:

- `--task <ID>` — execute only the specified task and its transitive dependencies
- `--dry-run` — display the execution plan without actually running tasks

Alias: `/spec implement`

```
/spec impl websocket                      # implement all tasks in order
/spec impl websocket --dry-run            # preview the execution plan
/spec impl websocket --task T-003         # implement only T-003 and its deps
/spec implement auth-v2                   # alias
```

Progress is reported through the `spec_task_update` tool, which creates-or-updates the session tracker task for each spec task (seeded from the spec's PLAN.md entry) as the runner works through the plan.

5.16 /spec tasks <spec-id>

Generate TASKS.md containing an ordered task list derived from the existing PLAN.md, plus quickstart.md with key validation scenarios derived from SPEC.md requirements.

```
/spec tasks websocket
```

5.17 `/spec jtbd <spec-id> [--force] [--agent <name>]`

Perform a Jobs-To-Be-Done analysis of an existing spec's `SPEC.md`, writing the result to `specs/<spec-id>/JTBD.md`.

The analysis extracts jobs using the JTBD framework:

- **Job statement** — "When <situation>, I want to <motivation>, so I can <expected outcome>."
- **Job type** — functional, emotional, or social
- **Performer** — who is hiring the product
- **Related requirements** — FR/NFR IDs that trace to this job
- **Success signals** — observable indicators the job is being fulfilled

The output also includes an out-of-scope section and a coverage matrix mapping requirements to jobs.

Flags:

- `--force` — overwrite an existing `JTBD.md`
- `--agent <name>` — dispatch to a specific agent (default: explore agent)

```
/spec jtbd websocket
/spec jtbd websocket --force
/spec jtbd auth-v2 --agent architect
```

5.18 `/spec feedback <spec-id> <note>`

Append a production feedback note to `specs/<spec-id>/FEEDBACK.md`. Notes are advisory — they are surfaced during `/spec plan` regeneration but do not block validation or status transitions.

Use this to capture production metrics, incident reports, or user feedback that should inform future plan iterations.

```
/spec feedback websocket Users report intermittent disconnections when switching networks
/spec feedback auth-v2 Token refresh fails silently when the server clock drifts by more than 5 s
```

5.19 `/spec govcreate <specid> <content-ref> <target-folder> [--language <lang>] [--type <type>] [--stack <name>] [--github | --gitlab] [--force]`

Create a project from an architecture document in one step: the command acquires the referenced documentation, extracts the architecture structure, authors a draft `SPEC.md` + `PLAN.md` + `TESTPLAN.md` into `<target-folder>/specs/<specid>/`, and scaffolds the project with the `/new` engine.

Arguments are positional and strictly ordered — all three positionals first, then the flags:

- `<specid>` — the new spec identifier. Alphanumeric with hyphens or underscores only and must not start with `--`; a path-traversal name is rejected before any filesystem access.
- `<content-ref>` — location of the documentation: a web URL to crawl, or a local file/folder path.
- `<target-folder>` — the project folder to create or reuse. The spec is written to `<target-folder>/specs/<specid>/`.

Flags after the positionals: `--language` (default `rust`), `--type` (default `cmdline`; one of `library`, `cmdline`, `tui`, `gui`, `webapp`), `--stack <name>` (a framework stack for the language), `--github` / `--gitlab` (mutually exclusive remote hosting), and govcreate-specific `--force` (overwrite an existing `<target-folder>/specs/<specid>/`; refused otherwise).

Behaviour, in order:

1. **Classify and acquire the content reference.** A URL runs a bounded, same-domain crawl that honours `robots.txt`, refuses private/loopback targets (SSRF protection) and non-HTTP(S) schemes, and never follows symlinks. A local file must be a supported document format (Markdown/text, PDF, DOCX, ODT, XLSX, ODS, PPTX, ODP, CSV, HTML, EPUB); a local folder is walked recursively for supported files. Acquisition is bounded by page, depth, character, and deadline caps and reports the first cap reached in the run notes.
2. **Extract the architecture structure** — components, interfaces, data stores, external dependencies, and relationships — from the gathered text, falling back to a deterministic per-source structure when the model output is not usable.
3. **Author the spec** with status `draft` frontmatter recording the source reference, `generated_by: govcreate`, and the scaffold options, then **scaffold the project** via the `/new` engine in the target folder.

Progress streams live in the message window and log panel while the run is in flight — a single assistant message is refreshed in place with one `[ .. ]/[ ok ]/[fail]` ASCII line per stage, and a terminal report summarises the outcome; pressing Escape while a run is live cancels its `CancellationToken`. Failure reporting is explicit: an invalid spec ID or flag shows the usage block with the cause; a blocked or unreadable content reference explains why (SSRF, robots, unreadable/unsupported/escaping file); an empty corpus or unusable model output stops before writing anything and names the cause; and an existing spec directory refuses unless `--force` was passed.

The same pipeline is available from the shell as `ragent spec govcreate <specid> <content-ref> <target-folder> [flags]` — it shares the TUI parser, resolves the model from `--model` or the stored `selected_model` (neither configured -> exit 2), prints the staged progress and final report to stdout, and exits non-zero on failure.

```
/spec govcreate payments-arch https://docs.provider.gov/payment-service ./payments-svc
/spec govcreate records-arch ./arch-records/records-hld.pdf ./records --language python --type li
/spec govcreate billing-arch ./doc/sad.md ./billing-svc --language rust --type cmdline --stack ax
/spec govcreate billing-arch ./doc/sad.md ./billing-svc --force   # overwrite an existing spec di
```

5.20 `/spec delete <spec-id> [--yes]`

Delete a spec directory from the workspace. The `--yes` flag skips the confirmation prompt.

```
/spec delete websocket --yes
/spec delete old-feature --yes
```

Spec file write semantics

All spec files are written atomically: content is written to a uniquely named temporary file, flushed and synced, then renamed over the target — so a crash mid-write can never leave a truncated `PLAN.md` or `SPEC.md`. Auxiliary review files follow a clear-on-empty rule: if `REVIEW.md` or `FEEDBACK.md` content is empty, the file is DELETED rather than replaced with an empty file, so the "file exists but says nothing" state (which would surface meaningless review notes in plan prompts) cannot occur.

---

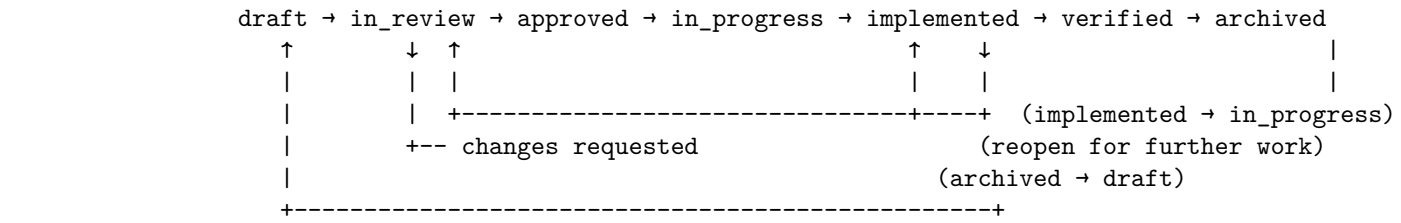
6. Lifecycle status and transitions

Specs progress through seven lifecycle stages. Transitions are enforced — you cannot skip stages.

Status values

Status	Description
`draft`	Initial draft state
`in_review`	Under review by stakeholders
`approved`	Approved and ready for implementation
`in_progress`	Implementation in progress
`implemented`	Code implementation complete
`verified`	Verified as correct and complete
`archived`	Archived, excluded from active queries

Transition graph



From	Allowed transitions
`draft`	`in_review`
`in_review`	`draft`, `approved`
`approved`	`in_progress`
`in_progress`	`implemented`
`implemented`	`verified`, `in_progress` (reopen)
`verified`	`archived`
`archived`	`draft` (reactivate)

Note: `in_review` can go back to `draft` (changes requested) or forward to `approved` (sign-off). Archived specs can be reactivated back to `draft`, and `implemented → in_progress` reopens an implemented spec for further work. The table is exactly what `SpecManager::allowed_transitions` permits; any other pair is refused by name before the file is touched (`invalid transition`).

Transition gating

When SDD flags are enabled, additional gates may block transitions:

- **Clarification markers** — if `sdd.clarification_markers` is enabled, transitioning to `approved` is blocked when unresolved `[NEEDS CLARIFICATION]` markers exist in the spec.
- **Phase -1 gates** — if `sdd.phase_minus_one_gates` is enabled, transitioning to `in_progress` is blocked until all pre-implementation gate checkboxes in `PLAN.md` are checked.

---

7. Task management

Tasks within a `PLAN.md` have their own status tracking.

Task status values

Status	Description	Symbol (coverage report / task list)
`pending`	Task not yet started	`[wait]`
`in_progress`	Task actively being worked on	`[sync]`
`completed`	Task completed	`[ok]`
`blocked`	Task blocked or deferred	`[stop]`

The symbols come from `TaskStatus::symbol()` and are shared by the `/spec coverage` report, the `spec_coverage` agent tool, and the TUI task listing, so status rendering is consistent everywhere.

Task table structure

The `PLAN.md` task table uses these columns:

Column	Description
ID	Task identifier (T-001, T-002, …)
Title	Human-readable task title
Requirement	Linked requirement IDs (FR-007, NFR-003, …)
Effort	Size estimate: S, M, or L
Priority	Critical, High, Medium, or Low
Status	Task status: Pending, in_progress, completed, or blocked (initially Pending)
Dependencies	Task IDs that must complete first (an inclusive range such as `T-001-T-014` expands to every spanned ID)

Dependency ordering

The implementation runner (`/spec impl`) resolves dependencies topologically and executes tasks in order. Tasks with no dependencies run first; tasks blocked by incomplete dependencies are deferred.

A Dependencies cell may list individual IDs (`T-001, T-003`), an inclusive range (`T-001-T-014`), or a mix. A range is expanded into every spanned ID, accepting an en dash (–), an em dash (—), an ASCII hyphen (-), or the words `to` / `through` between well-formed endpoints, with zero-padding preserved from the range's start ID. Before range expansion a cell like `T-001-T-014` was read as a single unknown ID, so the dependency was dropped and the final verification task was scheduled first instead of last.

Session tracker tasks (`spec_task_update`)

`/spec impl` does not pre-create session tracker tasks. The `spec_task_update` tool creates-or-updates the session tracker task itself when progress is reported: it matches existing tracker tasks on `spec_id`/`spec_task_id` metadata, maps the spec task status onto the tracker status, and seeds new tracker tasks from the spec's `PLAN.md` entry (with an `in_progress` instruction) so `/spec impl` progress stays visible without pre-populated rows. The `/spec impl` dispatch prompts instruct the agent to mark each tracker task `in_progress` BEFORE starting work on it.

Automatic task completion after writes

When an agent turn writes files into the active spec's own directory (`specs/<id>/`) and the session has an active spec, any task currently marked `in_progress` is automatically advanced to `completed` (with its completion timestamp recorded). This heuristic is deliberately guarded by `writes_in_spec_dir`: file-writing tools (`Write`, `Edit`, `multiedit`, `patch`, `apply_patch`, `create`, `append_to_file`) whose target path resolves OUTSIDE the spec directory (docs, scratch files, snapshots) never complete spec tasks, so editing unrelated files cannot silently corrupt `PLAN.md`. To mark tasks complete explicitly — or to mark a task done whose file lives outside the spec directory — use `/spec task <spec-id> <task-id> completed`.

Task completion timestamps

When a task is marked `completed` (via `/spec task` or `spec_task_update`), its completion time is written into the `PLAN.md` Status cell as RFC 3339 UTC, e.g. `| completed (2026-11-08T10:12:00Z) |`. No extra column is added, so 6- and 7-column task tables keep parsing; a bare `completed` (older plans) or an unparseable stamp reads as completed with no time. Moving the task to any other status drops the stamp. `/spec task <spec-id> <task-id>` shows the time in the local time zone; the `spec_read` tool reports it in UTC.

Persistence guarantees

The `SPEC.md`/`PLAN.md` round-trip preserves everything the writer does not own. The defects below were found in a full review of the spec system and are fixed; they are listed because each one silently rewrote part of a spec file.

Frontmatter

- A comment line (`# ...`) inside a `reviewers`, `research` or `audit` list is not a top-level key, so it no longer truncates the list, and the writer no longer copies the orphaned items into the regenerated block (`io::is_frontmatter_key_line`).
- The block closes only on a line that is exactly `---`, so a `---` inside a value does not cut it short.
- Keys the lifecycle manager does not model (`research:` linkage, custom keys) are preserved on every save, the audit trail is read back rather than regenerated empty, and multi-line `research:` lists parse.

Task table

- Empty cells stay empty, so a blank Requirement cell does not shift the columns after it.
- The em/en dash placeholders the row formatter writes read back as "none". Before, every task update gave an unlinked task a requirement and a dependency literally named with a dash.
- A status update rewrites the matching rows in place, preserving the trailing newline, instead of replacing everything between the task table and the next heading.
- A rewrite touches exactly what the plan parser reads as tasks: the first task header in the Tasks section (a legacy `| Task ID | Title |` header is normalised to seven columns) and data rows with at least six cells. An unrelated table that merely mentions a task ID, such as a three-column review table, is left untouched.

Status writes and discovery

- `/spec impl` reports a failed write to `implemented` as an error and leaves `SPEC.md` at its previous status, instead of reading the mutated in-memory status back as success.

- `/spec status` reports the transition it performed. It previously read the status after the transition and printed `from X to X`.
- Deleting the active spec deactivates it, and `/spec deactivate` invalidates the cached spec section of the system prompt.
- One unreadable spec is reported and skipped; it no longer aborts discovery for every other spec.
- `/spec impl` honours `sdd.phase_minus_one_gates` when moving a spec to `in_progress`.

---

8. The SDD workflow

The Spec-Driven Development (SDD) workflow separates specification from planning, allowing for review and clarification between stages.

SDD workflow stages

```
1. /spec specify <name> <feature>
       ↓
   SPEC.md created with [NEEDS CLARIFICATION] markers
       ↓
2. Review SPEC.md, resolve clarification markers
       ↓
3. /spec validate <name>
       ↓
   Validation passes
       ↓
4. /spec plan <name> <tech-context>
       ↓
   PLAN.md created from SPEC.md + technology context
       ↓
5. /spec tasks <name>
       ↓
   TASKS.md + quickstart.md created
       ↓
6. /spec status <name> approved
       ↓
7. /spec impl <name>
       ↓
   Tasks executed in dependency order
```

SDD vs. quick-create workflow

Quick-create (`/spec create`) generates everything in one step — ideal for small features where the specification is straightforward and the technology choices are obvious.

SDD workflow (`/spec specify` then `/spec plan`) separates the stages — ideal for larger features where requirements need review and clarification before planning, or where technology choices need careful consideration.

`/spec plan` vs `/spec update`

Both regenerate `PLAN.md`, but they differ:

- **`/spec plan <id> <tech-context>`** — generates the plan from `SPEC.md` using a technology context argument. Use this when you want to guide technology choices. This is the SDD stage-2 command.
- **`/spec update <id>`** — regenerates the plan from an edited `SPEC.md` without a technology context. Use this after manually editing `SPEC.md` to add or modify requirements. Also regenerates `TESTPLAN.md`.

---

9. SDD configuration

SDD capabilities are gated by configuration flags in `ragent.json` under the `sdd` key. All flags default to `false` (opt-in).

```
{
    "sdd": {
        "clarification_markers": true,
        "quality_checklists": true,
        "constitution": false,
        "phase_minus_one_gates": true,
        "branch_per_spec": true,
        "research_artifacts": true,
        "data_model": true,
        "contracts": true,
        "quickstart": true,
        "test_first_ordering": false,
        "consistency_checks": true,
        "amendment_process": false,
        "feedback_loop": true
    }
}
```

Flag reference

Flag	Effect
`clarification_markers`	Enable `[NEEDS CLARIFICATION]` marker detection in validation
`quality_checklists`	Embed quality checklists in spec and plan templates
`constitution`	Generate and parse `CONSTITUTION.md` architectural-principles artifact
`phase_minus_one_gates`	Enable Phase -1 pre-implementation gate validation
`branch_per_spec`	Create a git branch (`spec/<specname>`) per spec on `/spec specify`
`research_artifacts`	Link research artifacts into SPEC.md frontmatter
`data_model`	Generate `data-model.md` during `/spec plan`

Flag	Effect
`contracts`	Generate `contracts/` directory during `/spec plan`
`quickstart`	Generate `quickstart.md` validation scenarios
`test_first_ordering`	Enforce test-first file creation ordering in plans
`consistency_checks`	Run ambiguity, contradiction, and gap consistency checks
`amendment_process`	Enable constitutional amendment process with dated changelog
`feedback_loop`	Enable production feedback loop (`FEEDBACK.md` surfacing)

---

10. Validation details

`/spec validate` runs a series of checks on `SPEC.md` (and optionally `PLAN.md`). Issues are categorised by severity and category.

Severity levels

Severity	Description
`Error`	Must fix — blocks progression
`Warning`	Should fix — may indicate a problem
`Info`	Informational — no action required

Issue categories

Category	Description
`EarsSyntax`	Requirement does not match any EARS template pattern
`MissingSection`	Required section (e.g. `## Requirements`) is missing
`InvalidStatus`	YAML frontmatter status is not a valid SpecStatus
`Numbering`	Requirement numbering has gaps or duplicates
`Plan`	PLAN.md is missing or incomplete
`Structure`	Spec structure does not follow expected layout
`Clarification`	Unresolved `[NEEDS CLARIFICATION]` markers found
`Ambiguity`	Vague terms or undefined acronyms detected
`Contradiction`	Opposing requirement claims detected
`Gap`	Missing testable criteria or coverage gaps
`PhaseMinusOneGate`	Pre-implementation gate checkboxes not checked

Validation categories gated by SDD flags

When the corresponding SDD flag is `false`, the check is skipped:

Check	Gated by
Clarification markers	`sdd.clarification_markers`
Ambiguity detection	`sdd.consistency_checks`
Contradiction detection	`sdd.consistency_checks`
Gap analysis	`sdd.consistency_checks`
Phase -1 gates	`sdd.phase_minus_one_gates`
Constitution	`sdd.constitution`

---

11. Implementation orchestration

`/spec impl` reads the `PLAN.md`, parses the task table, resolves dependency order, and drives the agent through each task.

How it works

1. **Parse** — reads `PLAN.md` and extracts the task table into structured `PlanTask` objects with IDs, titles, requirements, effort, priority, and dependencies.
2. **Resolve order** — topologically sorts tasks by dependencies. Tasks with no dependencies run first; tasks blocked by incomplete dependencies are deferred.
3. **Build prompts** — constructs a per-task prompt that includes the task title, description, linked requirements, and the spec's SPEC.md content.
4. **Execute** — injects each task prompt into the agent session one at a time. The agent implements the task, and the runner moves to the next.
5. **Track progress** — emits progress updates showing completed, skipped, and blocked task counts.

Dry-run mode

`--dry-run` displays the execution plan without running any tasks. This shows the resolved execution order, milestone groups, and effort summary.

```
/spec impl websocket --dry-run
```

Single-task mode

`--task <ID>` executes only the specified task and its transitive dependencies. This is useful for re-running a failed task or implementing a specific subset.

```
/spec impl websocket --task T-003
```

Milestone groups

The runner identifies milestone groups from the plan — clusters of tasks that share a milestone deliverable. This helps track progress at a higher level than individual tasks.

---

12. Research integration

Specs can be linked to research artifacts using `--from-research`. This creates a two-way linkage:

1. The `SPEC.md` YAML frontmatter includes a `research:` field naming the research artifact.
2. A `## Related Research` section is added to the `SPEC.md` body.
3. The research artifact's `INDEX.md` is updated to reference the spec.

Linking a spec to research

```
/spec create websocket Add real-time collaborative editing --from-research realtime-collab
/spec specify auth-v2 Implement OAuth2 with PKCE --from-research oauth-study
```

Configuration

The `sdd.research_artifacts` flag in `ragent.json` enables this feature:

```
{
    "sdd": {
        "research_artifacts": true
    }
}
```

---

13. Branch-per-spec

When `sdd.branch_per_spec` is enabled, `/spec specify` creates a dedicated git branch named `spec/<specname>` to isolate spec work.

```
{
    "sdd": {
        "branch_per_spec": true
    }
}
```

The branch creation is tolerant — if the working directory is not a git repository or the branch already exists, spec creation proceeds without branching.

---

14. JTBD analysis

`/spec jtbd` performs a Jobs-To-Be-Done analysis on an existing spec, extracting the underlying "jobs" the feature is hired to do.

Output structure

The `JTBD.md` file contains:

1. **YAML frontmatter** with `status: draft`
2. **Overview** — one or two sentence summary of the spec
3. **Jobs** — each job with:

- Job statement using the grammar: "When <situation>, I want to <motivation>, so I can <expected outcome>."
- Job type: functional, emotional, or social
- Performer: who is hiring the product
- Related requirements: FR/NFR IDs that trace to this job
- Success signals: observable indicators the job is fulfilled

4. **Out-of-Scope Jobs** — jobs explicitly rejected or deferred
5. **Coverage Matrix** — table mapping each FR/NFR to the job(s) it supports

Example

```
/spec jtbd websocket
/spec jtbd websocket --force
/spec jtbd auth-v2 --agent architect
```

The `--force` flag overwrites an existing `JTBD.md`. The `--agent <name>` flag dispatches the analysis to a specific agent instead of the default explore agent.

---

15. Production feedback

`/spec feedback` appends advisory notes to `FEEDBACK.md`. These notes are surfaced during `/spec plan` regeneration — the agent is instructed to consider each note and add or adjust tasks to address identified issues.

Workflow

1. Capture a production issue or insight:

```
/spec feedback websocket Users report intermittent disconnections when switching networks
```

2. During the next plan regeneration, the feedback is surfaced:

```
/spec plan websocket Use tokio-tungstenite with automatic reconnection and connection state track
```

The agent receives the `FEEDBACK.md` content and is instructed to address each note by adding or adjusting tasks.

Configuration

The `sdd.feedback_loop` flag enables this feature:

```
{
    "sdd": {
        "feedback_loop": true
    }
}
```

---

16. End-to-end examples

Example 1: Full SDD workflow for a new feature

```
# Step 1: Specify the feature
/spec specify websocket Add a real-time collaborative editing feature using WebSockets

# Step 2: Review the generated SPEC.md, resolve [NEEDS CLARIFICATION] markers
# (manually edit specs/websocket/SPEC.md)

# Step 3: Validate
/spec validate websocket

# Step 4: Generate the plan with technology context
/spec plan websocket Use tokio-tungstenite for WebSocket server, Redis for pub/sub

# Step 5: Generate task list and quickstart
/spec tasks websocket

# Step 6: Check coverage
/spec coverage websocket

# Step 7: Move through the lifecycle
/spec status websocket in_review
/spec status websocket approved

# Step 8: Preview the implementation plan
/spec impl websocket --dry-run

# Step 9: Implement
/spec impl websocket

# Step 10: Verify and archive
/spec status websocket implemented
/spec status websocket verified
/spec status websocket archived
```

Example 2: Quick-create for a small feature

```
# Create everything in one step
/spec create csv-export Add CSV export for all list views

# Validate
/spec validate csv-export

# Implement
/spec impl csv-export
```

Example 3: Incremental feature addition

```
# Add a new feature to an existing spec
/spec add websocket Add presence indicators showing who is currently viewing each document

# Validate the updated spec
/spec validate websocket

# Regenerate the plan from the updated SPEC.md
/spec update websocket

# Implement the new tasks
/spec impl websocket --task T-008
```

Example 4: Research-driven spec

```
# First, run research on the topic
/research create oauth-study "OAuth2 PKCE flow for desktop applications"

# Then create a spec linked to the research
/spec specify auth-v2 Implement OAuth2 with PKCE --from-research oauth-study

# Validate, plan, and implement
/spec validate auth-v2
/spec plan auth-v2 Use oauth2 crate with PKCE, store tokens in SQLite
/spec impl auth-v2
```

Example 5: JTBD analysis of an existing spec

```
# Analyze the jobs a spec is hired to do
/spec jtbd websocket

# Review the generated JTBD.md
# (manually read specs/websocket/JTBD.md)

# Force regeneration after spec changes
/spec jtbd websocket --force
```

Example 6: Production feedback loop

```
# Capture a production issue
/spec feedback websocket Users report intermittent disconnections when switching networks

# Regenerate the plan with feedback surfaced
/spec plan websocket Use tokio-tungstenite with automatic reconnection

# The new plan will include tasks addressing the feedback note
/spec impl websocket
```

Example 7: Filtering and searching specs

```
# List all approved specs
/spec list --status approved

# Find specs about authentication
/spec search authentication

# List all specs including archived
/spec list --all

# Filter by ID prefix
/spec list --prefix auth
```

Example 8: Task management

```
# List all tasks in a spec
/spec task websocket

# Mark a task as in progress
/spec task websocket T-003 in_progress

# Mark a task as completed
/spec task websocket T-003 completed

# Mark a task as blocked
/spec task websocket T-005 blocked

# Implement a single task with its dependencies
/spec impl websocket --task T-005
```

---

17. Tips for good specs

- **Use specific feature descriptions.** "Add a real-time collaborative editing feature using WebSockets with OT conflict resolution" is better than "Add editing".
- **Use the SDD workflow for complex features.** Separate specification from planning so you can review and clarify requirements before committing to a technology approach.
- **Resolve all clarification markers before planning.** Run `/spec validate` after editing `SPEC.md` to confirm no markers remain.
- **Provide meaningful technology context.** `/spec plan` produces better plans when the tech context specifies crates, patterns, and architecture decisions.
- **Link specs to research.** Use `--from-research` to ground specs in prior research, ensuring the specification reflects findings.
- **Use `/spec add` for incremental features.** Don't recreate a spec when adding a feature — use `/spec add` to extend it without disrupting existing requirements.
- **Capture production feedback.** Use `/spec feedback` to record issues and insights that should inform future plan iterations.

- **Run `/spec impl --dry-run` first.** Preview the execution order before committing to implementation.
- **Check coverage before implementing.** `/spec coverage` reveals requirements without linked tasks.
- **Use branch-per-spec for isolation.** Enable `sdd.branch_per_spec` to isolate spec work on a dedicated branch.
- **Validate early and often.** Run `/spec validate` after any spec edit to catch issues before they propagate.

---

18. LLM tool surface

Specs are also accessible to the agent as first-class LLM tools, so a coding agent can read and track specs without slash commands:

`spec_read`, `spec_list`, `spec_search`, `spec_task_update` and `spec_coverage`; their arguments and examples are in tools/spec-management.md.

---

19. Troubleshooting

Symptom	Likely cause	What to do
`unknown subcommand`	Typo or missing arguments	Run `/spec help` for the command reference
Spec not found	Invalid spec ID or spec doesn't exist	Run `/spec list` to see available specs
`invalid transition`	Trying to skip a lifecycle stage (or reopen a spec from a status that does not allow it)	Follow the transition graph in section 6; `implemented` may reopen to `in_progress`
`/spec list` prints every spec	An unknown flag or status used to be ignored	Fixed: an unknown flag or status is an error naming the valid values. Check the spelling
`/spec status` printed `from X to X`	The status was read after the transition	Fixed: it now reports the transition performed
A requirement or dependency named `-`	Older ragent wrote dash placeholders that read back as names	Fixed: dash placeholders read back as "none". Update the task once to rewrite the row
`blocked by clarification markers`	Unresolved `[NEEDS CLARIFICATION]` markers	Edit `SPEC.md` to resolve markers, then re-validate
`blocked by unchecked gates`	Phase -1 gate checkboxes not checked	Edit `PLAN.md` to check all gate checkboxes
`no tasks found`	`PLAN.md` missing or has no task table	Run `/spec plan` or `/spec create` to generate a plan
`/spec impl` produces no output	No pending tasks or all tasks completed	Run `/spec impl --dry-run` to check task status

Symptom	Likely cause	What to do
Branch not created	Not a git repo or branch already exists	The spec is still created; check `git branch`
`data-model.md` not generated	`sdd.data_model` not enabled	Enable the flag in `ragent.json`
`contracts/` not generated	`sdd.contracts` not enabled	Enable the flag in `ragent.json`
JTBD overwrites existing file	`--force` not specified	Re-run with `--force` to overwrite
Feedback not surfaced in plan	`sdd.feedback_loop` not enabled	Enable the flag in `ragent.json`

End of manual.